

SKILL-2WEEK

TASK-1

```
root@DESKTOP-QU0HVJI:~/OSSP# tree
```

```
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
│   └── my_shell
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c
```

```
9 directories, 19 files
```

```
root@DESKTOP-QU0HVJI:~/OSSP# |
```

TASK-2

```
root@DESKTOP-QU0HVJI: ~/t × + v
root@DESKTOP-QU0HVJI:~# cd pratical
root@DESKTOP-QU0HVJI:~/pratical# cd practical-1
root@DESKTOP-QU0HVJI:~/pratical/practical-1# cat main.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    char command[100];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);

        // Executes only if exec fails
        printf("Execution failed!\n");
        exit(1);
    }
    else {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
    }
}
```

```
    printf("Parent PID: %d\n", getpid());

    wait(NULL);

    printf("Child process completed.\n");
}

return 0;
}
root@DESKTOP-QU0HVJI:~/practical/practical-1# gcc main.c -o main
root@DESKTOP-QU0HVJI:~/practical/practical-1# ./main
Enter a Linux command: ls

Parent Process

Parent PID: 644
Child Process
Child PID : 645
Parent PID: 644
main main.c
Child process completed.
root@DESKTOP-QU0HVJI:~/practical/practical-1# |
```

